Web Development Foundations

Week 8 – "Async Time Matters"

Day 2: "Async Timing – Requests, Promises, & Loading"

1

Reconnect

Monday's key question:

- What happens now?
- What happens later?

Today we apply that same question to:

- loading
- requests
- future results
- JSON-shaped data

2

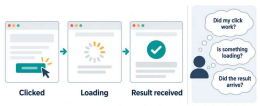
Timing Is A User Experience Issue

Async is not only a code issue.

It affects the user:

- Did my click work?
- Is something loading?
- Did the result arrive?
- Should I wait or try again?

"Good pages explain waiting!"



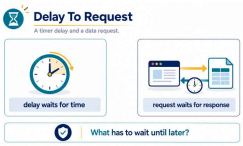
3

What Counts as Success Today

- trace what runs now and later
- explain why data is not instant
- describe `fetch` as starting a request
- describe a Promise as a future result
- read simple JSON structure before using it

4

From Delay To Request



Key Analogy:

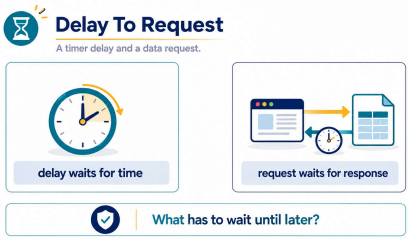
A delay waits for time.

A request waits for a response.

Both create the same beginner question:

- What can happen now?
- What has to wait until later?

5



Delay To Request

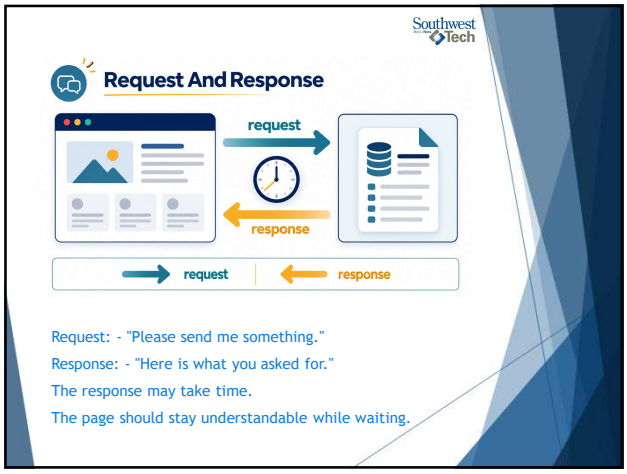
6


What We Will Use Today
request
response
`fetch()`
Promise
`async`
`await`
JSON
loading feedback

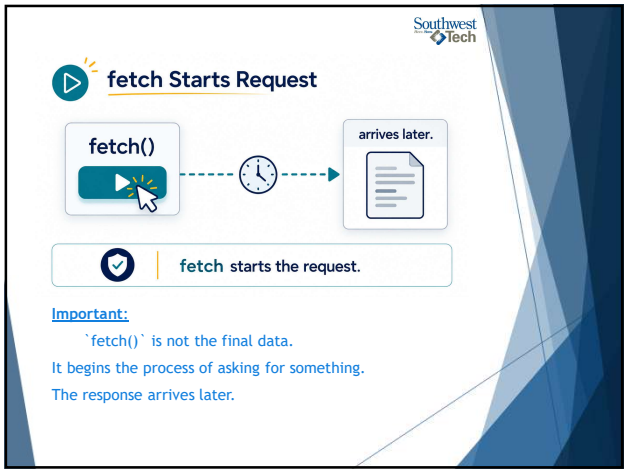
7


Request Toolbox
Request-shaped async concepts.
request
response
fetch()
Promise
async
await
JSON
loading feedback
Request-shaped async concepts.
Today's Toolbox

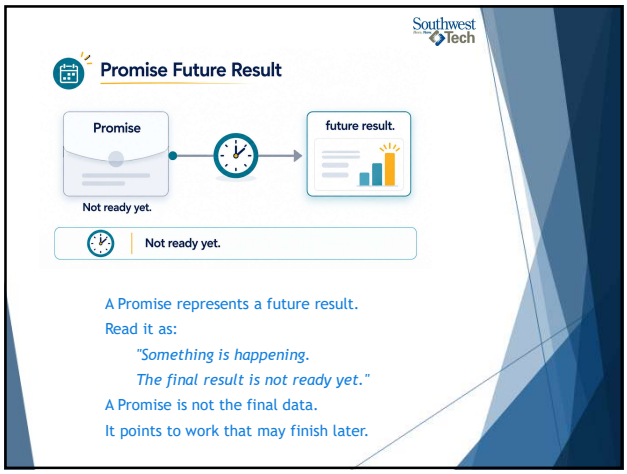
8


Request And Response
request
response
request
response
Request: - "Please send me something."
Response: - "Here is what you asked for."
The response may take time.
The page should stay understandable while waiting.

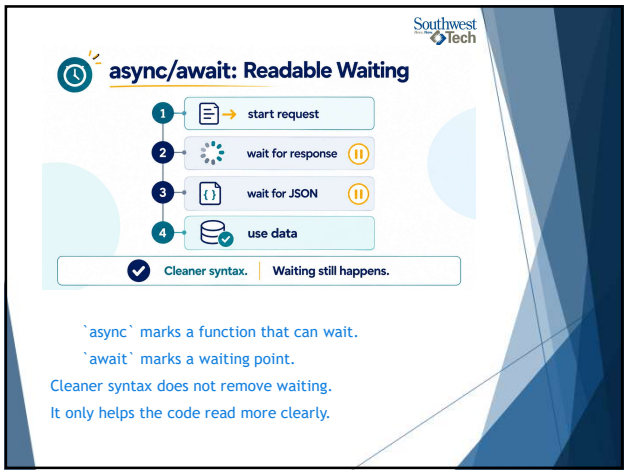
9


fetch Starts Request
fetch()
arrives later.
fetch starts the request.
Important:
`fetch()` is not the final data.
It begins the process of asking for something.
The response arrives later.

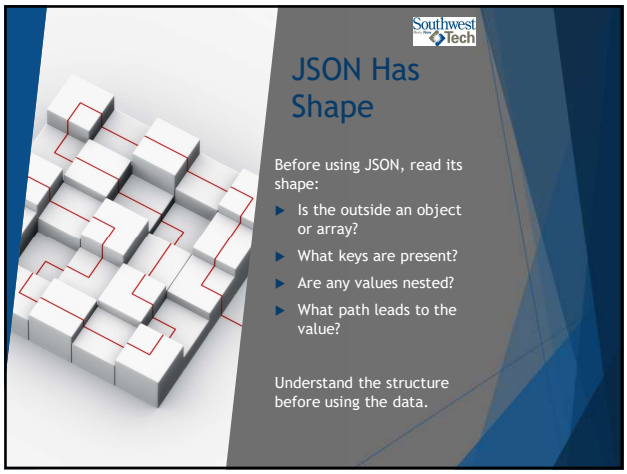
10


Promise Future Result
Promise
future result.
Not ready yet.
Not ready yet.
A Promise represents a future result.
Read it as:
"Something is happening.
The final result is not ready yet."
A Promise is not the final data.
It points to work that may finish later.

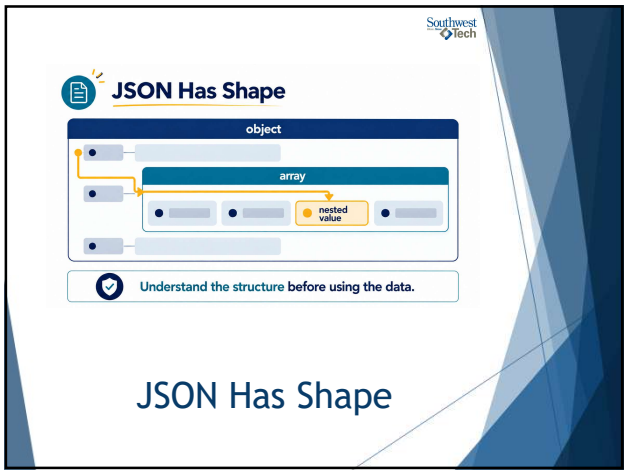
11


async/await: Readable Waiting
1 start request
2 wait for response
3 wait for JSON
4 use data
Cleaner syntax.
Waiting still happens.
`async` marks a function that can wait.
`await` marks a waiting point.
Cleaner syntax does not remove waiting.
It only helps the code read more clearly.

12



13



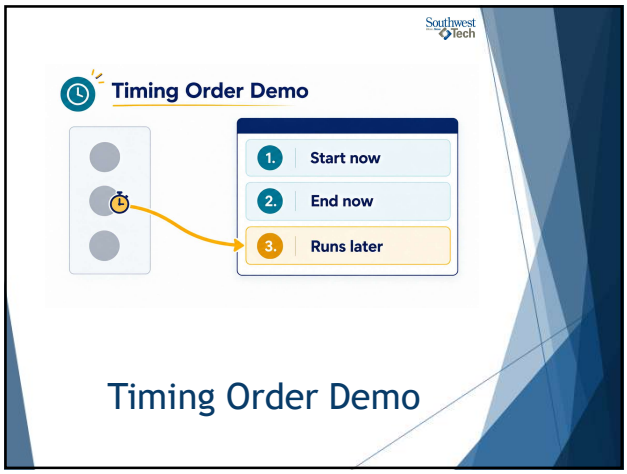
14



15



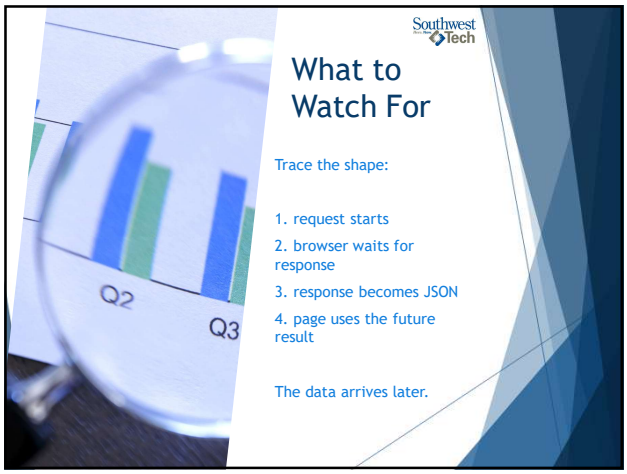
16



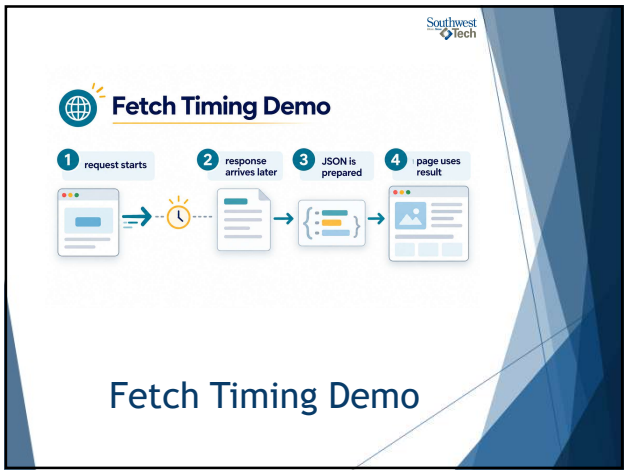
17



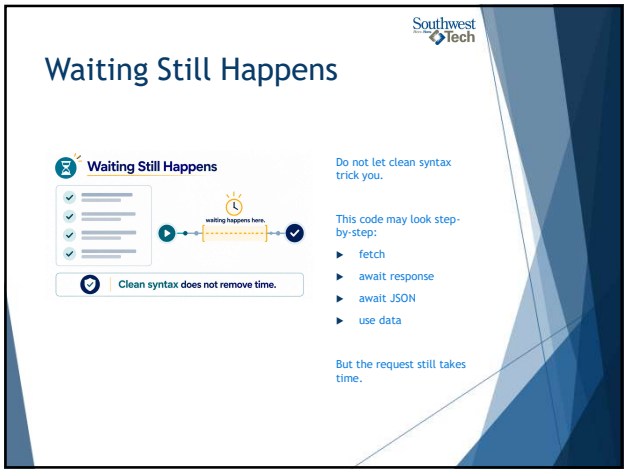
18



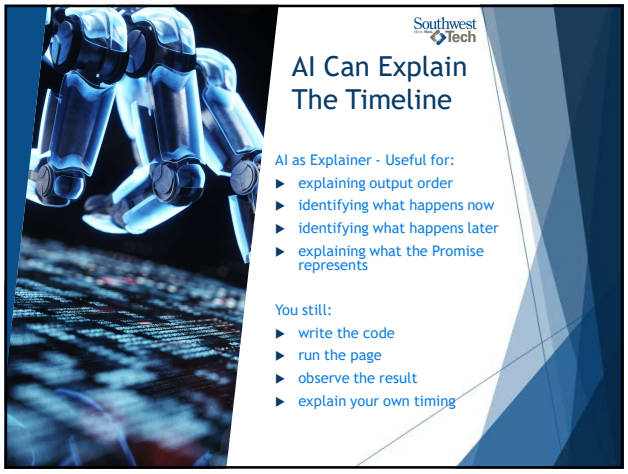
19



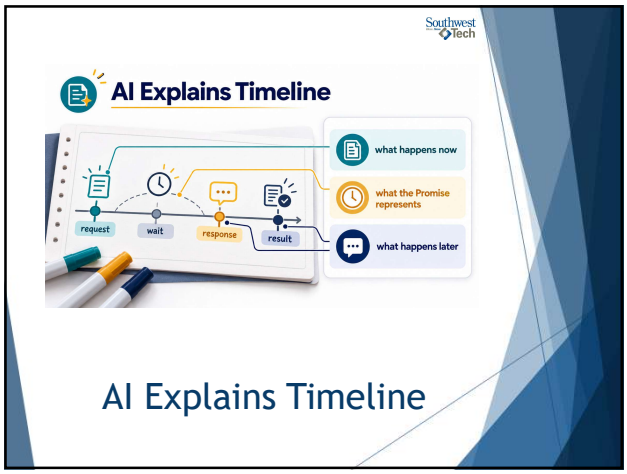
20



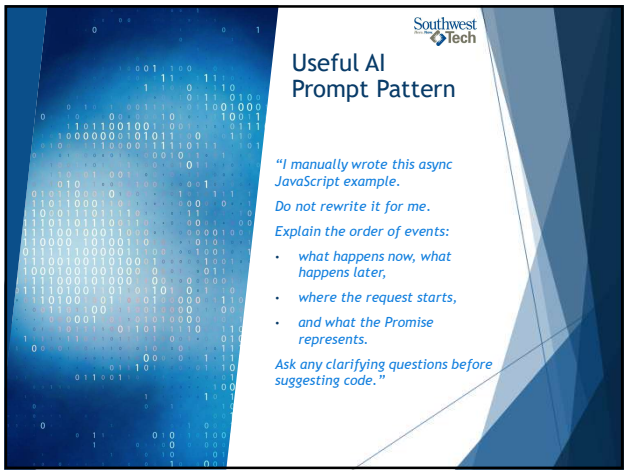
21



22



23



24

Useful AI Prompt Pattern

Context:	I manually wrote this async JavaScript example.
Constraint:	Do not rewrite it for me.
Timeline:	what happens now and what happens later.
Request:	where the request starts.
Promise:	what the Promise represents.

Ask one question before suggesting code.

Useful AI Prompt Pattern

25



Lab Refinement

In Lab 08:

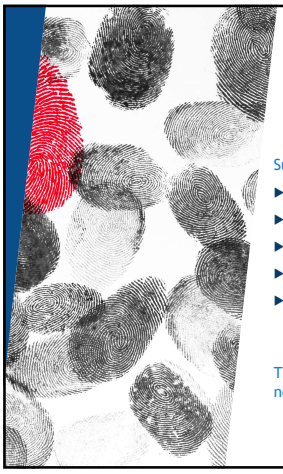
Improve your async behavior.

Choose at least one, (if you are ahead - do all of them):

- add a second timed interaction
- improve the waiting message
- make the sequence clearer
- structure the timed code better

Keep it explainable.

26



Evidence & Reflection

Submission evidence:

- timed behavior is visible
- waiting feedback is clear
- final result appears later
- code runs without errors
- reflection explains how timing changed your thinking

The timing should be intentional, not accidental.

27

Async Mental Model



Waiting still happens.

Async Mental Model

28



Closing Success Check

Async timing mental model:

- `fetch` starts a request.
- A Promise represents a future result.
- `async` / `await` makes waiting easier to read.

Waiting still happens.

Good pages explain the wait.

29



Thank you!

Have Fun!

30